Preprints.org

Article

Not peer-reviewed version

Surveying and Benchmarking Foundation Models for Time Series Classification

Juntao Fang , Ruichu Cai * , Yuming Liu , Shifeng Xie , Weilin Chen , Zijian Li , Keli Zhang , Lujia Pan , Themis Palpanas , Fuchun Sun

Posted Date: 24 September 2026

Keywords: time series classification; foundation models; transfer learning; in-context learning; empirical analysis

Preprints.org is a free multidisciplinary platform providing preprint service that is dedicated to making early versions of research outputs permanently available and citable. Preprints posted at Preprints.org appear in Web of Science, Crossref, Google Scholar, Scilit, Europe PMC, OpenAlex.

Copyright: This open access article is published under a Creative Commons CC BY 4.0 license, which permit the free download, distribution, and reuse, provided that the author and preprint are cited in any reuse.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

Disclaimer/Publisher's Note: The statements, opinions, and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions, or products referred to in the content.

Article

Surveying and Benchmarking Foundation Models for Time Series Classification

Juntao Fang [1], Ruichu Cai [1,*], Yuming Liu [1], Shifeng Xie [2,4], Weilin Chen [3], Zijian Li [1], Keli Zhang [2], Lujia Pan [2], Themis Palpanas [4] and Fuchun Sun [5]

[1] Guangdong University of Technology
[2] Huawei Noah's Ark Lab
[3] Peking University
[4] Université Paris Cité
[5] Tsinghua University
* Correspondence: cairuichu@gmail.com

Abstract

Foundation models enable the reuse of pretrained knowledge across time series classification (TSC) tasks, but transfer to a new task still requires target information to establish the task specific decision rule. Existing surveys mainly organize time series foundation models by architecture, pretraining objective, training data, or source modality, while the target classification procedure receives less attention. We instead organize foundation models for TSC into three transfer mechanisms: representation transfer, fine-tuning and adaptation, and in-context learning. Alongside the survey, we develop and release the TSC-FM Benchmark, a unified evaluation infrastructure covering 198 TSC datasets and multiple target settings. We use the benchmark to study how downstream classifiers, input settings, and task characteristics affect the assessment of pretrained models. The results show that the same pretrained representation can receive substantially different assessments under different downstream classifiers, that the best reported low shot model differs between univariate and multivariate settings, and that method groups lose accuracy at different rates as the number of classes increases. These findings suggest that transfer capability depends on the match between pretrained knowledge, target task structure, and the target procedure, rather than on the pretrained backbone alone. Future research should both expand the effective range of transfer and reliably identify when a target task falls outside that range.

Keywords: time series classification; foundation models; transfer learning; in-context learning; empirical analysis

1. Introduction

Time series classification (TSC) is a fundamental task in temporal data analysis, with applications in healthcare [1], industrial monitoring [2], human activity recognition [3], remote sensing [4], and many other domains [5,6]. TSC tasks can differ substantially in what defines a class: some depend on local waveform patterns, others on frequency, temporal order, or relations across channels [7,8]. Foundation models provide a way to reuse knowledge across such heterogeneous tasks instead of learning every representation from scratch.

Recent approaches include general time series pretraining [9,10], classification oriented pretraining [11,12], and reuse of models pretrained on vision [13], language [14], or tabular data [15–17]. These developments broaden the sources of reusable knowledge, but transfer to a new TSC task still requires target information to establish the task specific decision rule. The same pretrained knowledge may be used to fit a downstream classifier, update part of the model, or infer the target rule from labeled support.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

Existing surveys mainly organize time series models by architecture, learning objective, pretraining technique, training data, source modality, post training strategy, or representation learning paradigm [5,18–25]. Babu et al. [26] additionally review LLM based approaches to time series analysis. These perspectives explain how pretrained models are constructed, but less consistently characterize how the target classifier itself is established. This distinction matters because systems using similar or even identical pretrained backbones can differ in their use of target labels, optimization, task specific state, and inference procedure. Table 1 positions our survey relative to these complementary perspectives.

Table 1. Complementary organizing perspectives in related surveys. The table summarizes scope; it does not imply that a survey excludes topics outside its main emphasis.

Survey	Main scope	Organizing perspective
Fawaz et al. [5]	Deep time series classification	Network taxonomy and empirical architecture comparison
Foumani et al. [18]	Deep classification and extrinsic regression	Architectures, training methods, applications, and transfer learning
Ma et al. [20]	Time series pretraining	Pretraining techniques and transfer experiments
Zhang et al. [19]	Self-supervised time series learning	Generative, contrastive, and adversarial learning
Liang et al. [21]	Foundation models for time series	Architectures, pretraining, adaptation, and data modalities
Ye et al. [22]	Foundation-model reuse for time series	Pretraining modalities and their adaptation challenges
Kottapalli et al. [23]	Transformer-based time series foundation models	Architectures, output type, scale, and pretraining objective
Liu et al. [24]	Time series foundation models	Datasets, pretraining, and post-training
Trirat et al. [25]	Universal time series representations	Training data, architectures, and learning objectives
Babu et al. [26]	LLMs for time series, with classification experiments	LLM families, adaptation approaches, and an illustrative classification framework
This survey	**Foundation models for classification**	**Target classifier establishment, mechanism design, and comparison conditions**

We therefore organize foundation models for TSC around a simple question: **where is the target task learned when pretrained knowledge is transferred to a new classification problem?** This question shifts the focus from how the pretrained model is constructed to where target information enters the classification procedure, leading to three transfer mechanisms. *Representation transfer* constructs a decision rule on fixed pretrained features. *Fine-tuning and adaptation* update the pretrained model or selected components using target data. *In-context learning* (ICL) infers the target rule from labeled support without updating the main predictor for each target task. These mechanisms characterize how pretrained knowledge is connected to the target task rather than the backbone itself. Within each mechanism, we compare model origin, pretraining prior, learning objective, representation design, and channel interface.

Alongside the survey, we develop and release the TSC-FM Benchmark[1], a unified evaluation infrastructure covering 198 real world TSC datasets. The benchmark records complete classification configurations and supports analysis across target procedures, label budgets, input settings, and dataset characteristics. The empirical results show that conclusions can change with the downstream classifier, target setting, and class structure, suggesting that transfer capability depends on the match between pretrained knowledge, the target task, and the procedure used to establish the target decision rule.

[1] TSC-FM Benchmark: https://tsc-fm.dmirlab.com/

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

This survey makes four contributions:

(1) We organize foundation models for TSC into representation transfer, fine-tuning and adaptation, and in-context learning according to where the target task is learned.
(2) We provide a structured synthesis of 27 included method papers through 31 August 2026, comparing their pretraining, representation, channel handling, and target procedures.
(3) We develop and release the TSC-FM Benchmark, covering 198 real world datasets, and use it to examine how target configurations and task characteristics affect empirical conclusions.
(4) We summarize open problems in transferable representations, target adaptation, general and scalable ICL, reliable transfer, and reproducible evaluation.

Section 2 formulates transfer to new TSC tasks, Section 3 develops the three transfer mechanisms, Section 4 presents the TSC-FM Benchmark and empirical analysis, and Section 5 discusses challenges and future directions.

2. Background and Problem Formulation

2.1. Time Series Classification

Let a target task τ provide labeled examples

$$S_\tau = \{(X_i, y_i)\}_{i=1}^{n_\tau}, \qquad X_i \in \mathbb{R}^{T_i \times C_i}, \quad y_i \in \mathcal{Y}_\tau.$$

Here, T_i and C_i denote sequence length and channel count, and $\mathcal{Y}_\tau$ is the target class set. A classifier maps a query X_q to a prediction $\hat{y}_q \in \mathcal{Y}_\tau$. We use S_τ to denote the labeled target data available for establishing the classifier, whether they are used to fit a decision rule, adapt the model, or condition prediction at inference time. Validation use is recorded separately as an operation, while its labels count toward the total target-label budget. We denote any additional permitted unlabeled target observations by U_τ; this set can be empty. Access to unlabeled queries or query batches is specified separately by the evaluation protocol. Test labels are reserved for scoring and do not form part of the information available to the predictor.

Time series classification tasks can differ in several ways. The label space may vary in both the number and meaning of classes. Tasks can also differ in the temporal patterns or cross-channel relations that define those classes. In addition, sequence length, channel configuration, and input distribution may vary across datasets. These differences concern the label space, the classification rule, and the input space, respectively. Distinguishing them is important because cross-task variation cannot be reduced to differences in label symbols alone.

2.2. Transfer to New Classification Tasks

A foundation model provides reusable state θ learned from multiple datasets, domains, or generated tasks. This state may define an encoder g_θ, an initialization, a shared prediction function, or a predictor that uses labeled support examples. The target classification procedure combines this state with the available information from the target task. If class semantics or an external ontology are available, they should be treated as additional sources of information and distinguished from labeled target examples.

Two requirements arise when transferring to a new task. The model must preserve information relevant to the class structure, and the prediction procedure must connect that information to the target labels. A representation may preserve useful discriminative information while leaving the label mapping to a fitted classifier or labeled context. Conversely, a correct label mapping cannot recover information that has been lost in the representation. Transfer therefore depends on both the match between the representation and the target task and the construction of an appropriate decision rule.

The need for target label information can be illustrated by a simple label permutation argument. Let $\pi : \mathcal{Y}_\tau \to \mathcal{Y}_\tau$ be any permutation of the target labels. Replacing each label y with $\pi(y)$ leaves the target input distribution unchanged. A fixed pretrained predictor that observes neither labeled target

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

examples nor semantic information about the classes therefore receives the same input under the original and permuted tasks. It cannot determine which label assignment is correct. The correspondence between class structure and target labels is thus not identifiable from unlabeled inputs alone. This argument concerns label assignment rather than the recovery of class structure itself. Even when useful class structure is available, it must still be connected to the labels of the target task. Labeled examples, prototypes associated with class labels, or semantic descriptions can provide this information. It can then be used through parameter updates, in-context prediction, or nonparametric methods. More generally, label correspondence is only one part of task transfer. Other differences may arise from the label space, the classification rule, and the input distribution.

We include pretrained or meta trained systems with reported transfer across multiple TSC tasks and a traceable target classification procedure. The scope covers models designed for classification, general time series models with evaluated classification configurations, and systems that use foundation models for TSC. Domain specific models such as FORMED [27] enter the scope through their cross dataset classification and disclosed adaptation procedure. Data generators, studies focused exclusively on forecasting, and evaluation diagnostics provide supporting evidence rather than independent foundation classifiers.

2.3. Overview of Transfer Methods

All transfer methods combine pretrained knowledge with information available from the target task to produce predictions over $\mathcal{Y}_\tau$. We group existing approaches into three mechanisms.

In *representation transfer*, the pretrained encoder remains fixed, and labeled target examples are used to construct a classifier, prototype rule, or other decision function on the extracted representations. In *fine-tuning and adaptation*, target data are used to update the pretrained model or additional trainable modules involved in prediction. Training only a classifier on frozen representations is therefore considered representation transfer. In *in-context learning*, labeled support examples are provided as part of the input context, and prediction is performed without updating the model parameters for each target task.

Methods are classified according to how the pretrained model is used on the target task rather than by the backbone itself. The same backbone may therefore support different transfer mechanisms. For example, fine-tuning the pretrained model and training a classifier on its frozen representations belong to different mechanisms even when they use the same backbone. Preprocessing, retrieval, calibration, and support caching are treated as auxiliary operations and recorded separately from the transfer mechanism.

The corresponding methods are reviewed in detail in Section 3.

3. Transfer Methods for Time Series Classification

We organize existing methods by where the target task is learned: in a decision rule fitted on fixed representations, through updates to the pretrained model or adaptation modules, or from labeled support at inference time. Within each mechanism, we compare model origin, pretraining prior, learning objective, representation design, and channel interface. Table 2 summarizes representative configurations, while Figure 1 places selected developments in time.

As shown in Figure 1, the three mechanisms have developed in parallel rather than as successive stages. Representation transfer has expanded from general pretrained features to representations designed for classification, synthetic pretraining, cross modal reuse, and domain specific applications. Fine-tuning and adaptation have progressed from full model updates toward more selective changes to input interfaces, channel processing, and model components. ICL has emerged more recently, extending from temporal classification with labeled support to the reuse of tabular task priors and multivariate task inference. The timeline summarizes these broad developments rather than strict methodological lineages or differences in effectiveness.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

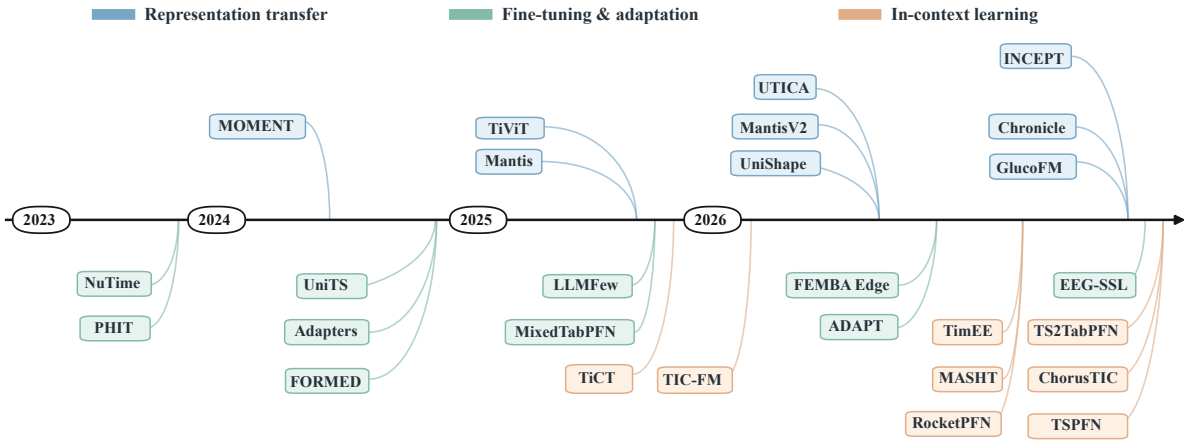


Figure 1. Timeline of representative developments in foundation models for time series classification through August 2026. Colors denote representation transfer, fine-tuning and adaptation, and in-context learning. Each method is placed according to the primary target classification procedure emphasized in its reported TSC evaluation. Systems with evaluated uses under multiple mechanisms are shown once for visual clarity; their additional configurations are recorded in Table 2.

Table 2. Representative classification configurations, organized by evaluated transfer mechanism. The source column combines model origin, prior data and learning objective; the next column describes representation and channel handling. Multiple mechanisms require supported, evaluated uses of the authors' system. Comparison baselines do not create additional assignments. NR: not reported in the cited paper.

System	Source prior and objective	Representation and channel interface	Target classifier and updated state	Mechanism
MOMENT [9]	General TSFM; Real mixed-domain Time Series Pile. Masked patch reconstruction	Patch Transformer; sequence embedding. Independent: encode each channel separately	Freeze encoder; fit and select one-versus-rest RBF-SVM	Representation
TiViT [13]	Vision reuse; Real image or image-text corpora. Inherited contrastive, masked, or self-distillation vision objectives	Rendered patch matrix; frozen ViT; pooled hidden features. Independent: render channels, then concatenate embeddings	Fit logistic regression; centroid and RF controls	Representation
GlucoFM [28]	CGM-domain pretraining; masked latent and temporal-dynamics prediction	Daily glucose grid with observation mask; slow-trend and short-deviation streams	Freeze encoder; fit logistic regression, including low-label settings	Representation
NuTime [29]	General TSFM; pooled real time series. BYOL-style self-supervision	Shape, mean and std window embeddings; Transformer. Concatenate and project channels	Fine-tune encoder and MLP head; add multivariate input projection	Adaptation
PHIT [30]	TSC-oriented; real UCR datasets grouped by domain. Dataset-identity pretext classification	H-Inception CNN; first three modules pretrained, three added for target learning; dataset-specific batch normalization. Univariate evaluation	Fine-tune all six modules and target classification layer; ensemble five models	Adaptation
UniTS [10]	General TSFM; Real multi-domain time series. Masked reconstruction across predictive and generative tasks	Patch tokens; time and variable self-attention Transformer. Joint: attend over time and variables	Fine-tune all weights, or learn prompt tokens, task tokens, and class representations	Adaptation
LLMFew [14]	Language reuse; Real language corpora. Inherited autoregressive language modeling	Per-channel temporal convolutions; LoRA-adapted decoder LLM and MLP head; multivariate fusion	Train the temporal encoder, attention LoRA, and MLP head	Adaptation
FORMED [27]	TSFM reuse; General time series, then labeled medical cohorts. Forecasting, then cross-dataset medical classification	Frozen TimesFM and shared decoding-attention classifier. Hybrid: channel-wise backbone, then shared attention	For a new target, learn channel embeddings and label queries; keep backbone and shared decoder frozen	Adaptation
EEG SSL adaptation [31]	BIOT or CBraMod reuse. Retain contrastive or masked reconstruction objective	Existing EEG backbone; adapt final encoder layer with unlabeled target data	Freeze adapted encoder, then fit labeled linear head	Adaptation
ADAPT [32]	TSC pretraining; 162 real classification datasets. Time and frequency span-masked reconstruction	Adaptive average pooling aligns sequence length and channel dimensions; separate time and frequency projections with a Transformer encoder	Fine-tune on each target classification dataset	Adaptation
MixedTabPFN [33]	TabPFN reuse; synthetic mixed tabular and time-series data. Classification and temporal-consistency objectives	Dilated temporal encoder, cross-modal embeddings, and temporal attention for mixed tabular and sequential inputs	Fine-tune embedding layers and attention masks; keep the TabPFN backbone frozen	Adaptation
FEMBA on the Edge [34]	EEG-domain pretraining; reconstruction with low-pass targets	Bidirectional Mamba over EEG patches; multichannel input	Fine-tune encoder and linear head; quantization for deployment	Adaptation

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

Table 2. Representative classification configurations, organized by evaluated transfer mechanism (continued).

System	Source prior and objective	Representation and channel interface	Target classifier and updated state	Mechanism
TiCT [12]	TSC-native; Synthetic KernelSynth mixture tasks. Episodic in-context classification	ResNet/Transformer encoder; contextual encoder-decoder and output attention. Univariate evaluation	Retrieve labeled neighbors as support; no target-weight update	In-context
TimEE [35]	TSC-native; Synthetic VARX classification tasks. PFN-style query negative log-likelihood	Patch encoder with temporal, cross-series, and in-context attention. Univariate main model; independent or learned-pooling extension	Supply labeled support in one forward pass; no per-dataset weight update	In-context
TIC-FM [36]	TSC-native; CauKer series, SCM tasks, and UCR-training-split adapter alignment. Contrastive encoder learning and synthetic episodic classification	Convolution tokenizer, ViT, MLP adapter and latent-memory Transformer. Univariate; separate multivariate adapter	Supply labeled support; no per-dataset deployment update	In-context
RocketPFN [15]	Tabular PFN; Synthetic tabular tasks. PFN classification	Grouped random ROCKET features and frozen TabPFN 2.5. Joint: kernels select channel subsets	Create random kernels and supply labeled feature context; no gradient update	In-context
TS2TabPFN [16]	Tabular PFN; Synthetic tabular tasks. PFN classification	Explicit temporal features and frozen TabPFN 2.5. Feature-dependent: per-channel or native multivariate extraction	Fit the feature transform and supply labeled context; no TabPFN weight update	In-context
MASHT [17]	Tabular PFN; Synthetic tabular tasks. PFN classification	MultiRocket and Hydra features with frozen TabPFN-3. Native multivariate MultiRocket and Hydra transforms	Fit transforms and scaler, then supply labeled context; no TabPFN weight update	In-context
ChorusTIC [37]	TSC-native; synthetic labeled multivariate tasks. Episodic classification	Dual-axis encoder; episode-consistent subchannel slots and context-derived feature calibration	Labeled context and query; no target-weight update	In-context
TSPFN [38]	TabPFN initialization, then labeled physiological pretraining. Query-label cross entropy	Temporal and channel embeddings within the PFN architecture	Labeled support and query; no target-weight update	In-context
Chronicle [39]	Joint text–TS pretraining from scratch; real text and TS corpora with KernelSynth augmentation. Autoregressive text and quantile forecasting objectives	Shared decoder-only Transformer; text tokens and channel-aware TS patches. Joint multivariate encoding; pooled hidden embeddings	UCR/UEA: freeze backbone and fit linear probe. Text–TS classification: freeze backbone and fit MLP head, or train LoRA and head with original backbone weights fixed	Multiple
Mantis [11]	TSC-native; Synthetic CauKer series. InfoNCE contrastive learning	Three-branch convolution tokenizer and Transformer; independent channel encoding and concatenation	Fit logistic regression or RF; optional end-to-end fine-tuning	Multiple
MantisV2 [40]	TSC-native; Synthetic CauKer series. InfoNCE contrastive learning	Refined convolution tokenizer, lighter Transformer, layer aggregation; independent channels and concatenation	Fit logistic regression or RF; optional end-to-end fine-tuning	Multiple
Adapters [41]	TSFM reuse; MOMENT masked reconstruction or Mantis contrastive pretraining. Inherited checkpoint priors	PCA/SVD, random projection, variance selection, or learned linear combiner compress channels; encode compressed channels independently	Fit linear head with fixed projection and frozen encoder; train combiner and head, optionally fine-tune backbone	Multiple
UniShape [42]	TSC-native; Real archive series; source labels partly used. Instance- and shape-prototype contrastive learning	Multiscale shape adapter and Transformer encoder. Univariate evaluation in cited paper	Fine-tune encoder and classifier; fit RF in held-out feature-transfer evaluation	Multiple
UTICA [43]	TSC-native; synthetic causal-DAG series. DINO crop matching, iBOT masked-patch self-distillation, and KoLeo regularization	Mantis-style convolution/statistical tokenizer and Transformer; teacher embeddings. Univariate encoder; multichannel aggregation NR	Fit linear classifier on frozen embeddings, or fine-tune full backbone and classifier	Multiple
INCEPT [44]	TSC-oriented EEG; unlabeled clinical recordings. Invariance-oriented learning	EEG encoder; linear readout for most frozen tasks, sequence head for sleep staging	Fit a frozen-encoder readout, or fine-tune encoder and head	Multiple

3.1. Representation Transfer

Representation transfer keeps the pretrained encoder fixed while constructing a target decision rule:

$$\hat{h}_\tau = \mathrm{Fit}\big(\{(g_\theta(X_i), y_i)\}_{i=1}^{n_\tau}\big),$$
$$\hat{y}_q = \hat{h}_\tau\big(g_\theta(X_q)\big).$$

The operator Fit includes learned classifiers and rules constructed from labeled reference examples. For a learned head,

$$\hat{\phi}_\tau = \arg\min_{\phi} \sum_{(X_i, y_i) \in S_\tau} \ell\big(h_\phi(g_\theta(X_i)), y_i\big).$$

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

Linear and nonlinear classifiers, prototypes, and nearest-neighbour rules differ in their decision boundaries and retained state. All use target labels to interpret fixed features. Target-derived feature transformations require an additional distinction below.

From general pretraining to representations for classification. MOMENT [9] provides an early example of using a general time series model for classification. Its pretrained encoder is kept fixed, and an RBF-SVM is trained on the extracted features. The encoder provides temporal representations, while the SVM learns the mapping to the target classes. This setting shows that features learned from general time series pretraining can support downstream classification, although the reported performance reflects both the encoder and the classifier. UniTS [10] also aims to learn broadly reusable time series representations, but its reported classification settings update model parameters and are therefore discussed as adaptation.

Mantis [11] places greater emphasis on representations learned for classification and evaluates both frozen representations and fine tuning. This line of work shifts the focus from whether pretrained features can be reused to what information they preserve for a classification task. UniShape [42] introduces objectives based on instances, shapes, and prototypes, while UTICA [43] reuses the Mantis tokenizer and encoder with a distillation objective based on crop consistency and local masked prediction. These methods encourage the encoder to preserve different aspects of temporal structure. ADAPT [32] and PHIT [30] provide further examples through reconstruction and dataset identification objectives, while their use for target adaptation is discussed in Section 3.2.

The value of such objectives depends on the target task. For example, removing amplitude variation may help recognize the same waveform across sensors, but it may also remove information when amplitude itself distinguishes the classes. This example illustrates a general tradeoff rather than a reported limitation of a specific method. The important question is therefore not only whether the pretraining objective produces useful features, but also which distinctions remain available in those features for future classification tasks.

Pretraining source and feature use. The development from Mantis [11] to MantisV2 [40] also illustrates changes in the source of pretraining data. Mantis uses mixed real time series, whereas MantisV2 uses synthetic data generated by CauKer [45] and introduces further changes to the architecture and inference procedure. CauKer generates the pretraining observations, while Mantis learns the representations that are later used by a target classifier. Changing the source data can broaden the temporal patterns seen during pretraining without changing how the target classifier is constructed.

Other methods extend representation transfer beyond time series pretraining. TiViT [13] converts time series into a visual representation, uses a frozen vision encoder, and trains logistic regression on the resulting features. Chronicle [39] jointly pretrains on language and time series and evaluates its representations with a linear probe. These methods broaden the source of pretrained knowledge, but they also introduce additional choices in how time series are represented before classification. Rendering, pooling, and feature aggregation can all affect which temporal information reaches the classifier. Studies based on forecasting models similarly show that representations learned for forecasting can contain information useful for classification [46]. These results demonstrate reuse in the reported settings, but they do not provide a common comparison among different sources of pretrained knowledge.

Domain specific models make this issue more concrete. INCEPT [44] learns EEG representations with objectives designed around invariance and evaluates both frozen classifiers and full fine tuning. Most of its frozen settings use a linear classifier, while sleep staging uses a sequence model as the prediction head. GlucoFM [28] separates slow glucose trends from short term variations and trains logistic regression on frozen representations for phenotype classification. Its experiments with limited labels vary the number of labeled subjects available to the classifier. These examples show that domain knowledge can guide what information is preserved in the representation, although the conclusions remain specific to the signals and tasks that were evaluated.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

Information retention and classifier access. Poor classification performance with frozen representations can arise for at least two reasons. The representation may fail to preserve information needed for the target task, or the information may still be present but difficult for the chosen classifier to recover from the available labels. A more expressive classifier can help in the second case, but it cannot recover information that has already been removed by the representation.

Auer et al. compare random forests, linear classifiers, and 1-NN on features obtained from forecasting models, while prompting and probing studies provide related evidence about the information available in pretrained representations [46,47]. When the features, labels, and model selection procedure are held fixed, consistent gains from a nonlinear classifier suggest that the linear classifier does not make full use of the available representation. By contrast, poor results across several classifiers do not show that the required information is absent. Limited labels or model selection may also explain the result.

Operations before the classifier must also be considered. Preprocessing, sequence length handling, pooling, and channel aggregation can change the information that reaches the final decision rule. Encoding each channel separately may leave relations between channels for the classifier to learn, whereas aggregation that removes channel identity may also remove useful information. LRG [48] provides a related example. Its MOMENT configuration keeps the encoder and logistic classifier fixed, but constructs the classifier input using local residual geometry derived from target labels. The encoder is therefore frozen, while the features given to the classifier still depend on the target task. Classifier capacity, feature construction, and source pretraining should therefore be considered separately. Section 4.4 compares configurations using the same backbone while accounting for differences in their feature pipelines.

Overall, representation transfer has moved from asking whether pretrained features can be reused to asking what information they preserve and how effectively that information can be used by a downstream classifier. General time series models, models trained for classification, synthetic pretraining, reuse across modalities, and domain specific models all provide evidence that frozen representations can support new classification tasks. However, their success depends on both the information retained in the representation and the classifier used to interpret it. Representation transfer is therefore most suitable when the pretrained encoder already preserves the information required by the target task and the remaining target learning can be handled by a separate decision rule.

3.2. Fine-Tuning and Adaptation

Adaptation uses labeled target data and, when permitted, additional unlabeled target data to update the pretrained model:

$$\theta_\tau = \mathrm{Adapt}(\theta, S_\tau, U_\tau),$$

where S_τ denotes the labeled target data and U_τ denotes any additional unlabeled target data available for adaptation. The adapted state may include backbone weights, adapters, prompts, or channel embeddings. A target head may also be trained jointly. If the fixed representation already separates the target classes, a new readout may be sufficient. If it does not preserve a distinction required by the target task, earlier parts of the model may need to be updated, provided that the input still contains the relevant information.

From pretraining across datasets to selective target updates. NuTime [29] and PHIT [30] are early examples of pretraining across datasets followed by target fine tuning. NuTime uses a self supervised objective based on BYOL [49] and then fine tunes the full model on the target task. PHIT instead trains the model to identify the source dataset of each time series during pretraining. NuTime represents each window using its normalized shape, mean, and standard deviation. The mean and standard deviation are embedded at different numerical scales. This design handles differences in amplitude while preserving these statistics. For multivariate classification, the model combines embeddings from different channels through an additional projection before the Transformer. These choices affect the information available to the model, while fine tuning adapts the representation to the target task.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

Later work explores different ways to adapt pretrained models. UniTS [10] evaluates both full fine tuning and prompt or task token tuning. Multivariate adapters [41] modify how channels are compressed or combined before they enter the backbone. FORMED [27] separates a shared medical classifier from channel embeddings and label queries that are learned for each target task. MixedTabPFN [33] illustrates adaptation at the input interface: it augments a frozen TabPFN backbone with temporal and cross-modal modules for mixed tabular and time-series inputs, and updates only the embedding layers and attention masks during target fine-tuning. These methods show that adaptation is not only a choice between freezing and fine-tuning the whole model. It can instead modify the parts of the system that mediate the mismatch between the pretrained model and the target input.

The part being updated determines what adaptation can change. An input adapter changes how the target data are presented to the backbone. Updating the backbone changes the representation itself. FORMED [27] first learns a shared decoding attention classifier on medical data. For a new target task, it keeps the backbone and shared classifier fixed and learns only the channel embeddings and label queries. This strategy relies on the knowledge learned during the earlier medical training stage. LLMFew [14] takes a different approach. It trains a temporal encoder, LoRA parameters in the attention layers, and an MLP head with a pretrained language model. Thus, the effect of adaptation depends not only on how many parameters are trained, but also on which parts of the model are updated.

Input handling during pretraining should be distinguished from target adaptation. ADAPT [32] allows time series with different sequence lengths and channel dimensions to be trained together, and supports downstream fine tuning. However, the transfer mechanism is determined by how the pretrained model is used on the target task, rather than by its pretraining interface. The same distinction applies to UniShape [42] and UTICA [43]. Both evaluate full model fine tuning, while UTICA also reports frozen representation transfer.

Supervision and deployment constraints. Adaptation can also differ in the target data it uses and the cost of updating and deploying the model. Dani and Liebe [31] use unlabeled target data to adapt the final encoder layer of BIOT or CBraMod. They then freeze the encoder and train a linear classifier with labeled target data. Their evaluation also controls the computation used for adaptation. FEMBA on the Edge [34] instead fine tunes both the EEG encoder and a linear classifier, and further studies quantized deployment. Although both methods use a linear classifier at the final stage, they are adaptation methods because the encoder is updated using target data.

Comparative studies show that the effect of adaptation depends on the backbone, target data, and the part of the model being updated. Nguyen et al. compare training only the classification head with LoRA adaptation for LaBraM and REVE, and report different results across backbones and subject groups [50]. NeuroAdapt-Bench [51] studies several forms of test-time adaptation that update normalization parameters, feature representations, or class prototypes. These results show that different adaptation strategies should be compared under the same target setting and against the same baseline.

Choosing what to adapt. The appropriate update depends on what the pretrained model already provides for the target task. If fixed features already separate the target classes, changing the classifier may be sufficient. Differences in input format or channel structure may require a projection or adapter. If the representation does not preserve a distinction needed for classification, the encoder itself may need to be updated. Domain shift may also motivate adaptation using unlabeled target data. These choices depend on the target setting and should not be treated as fixed rules.

Existing studies show that adaptation can be applied at different levels, from input projections and target-specific components to the backbone itself, and that target adaptation does not always require updating the full model. However, no single update scope works consistently across backbones and target tasks. Updating fewer parameters can reduce the extent of model change, but does not necessarily reduce the need for labels, validation, or optimization. A remaining challenge is therefore to determine which parts of the model should be updated when only limited target data are available.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

3.3. In-Context Learning (ICL)

ICL predicts a query using labeled support:

$$p_\theta(y_q \mid X_q, S_\tau), \qquad \theta \text{ fixed for task } \tau.$$

Support supplies the observed input-label associations. The predictor must infer a rule that explains them and applies to the query, using its pretrained task prior. Eliminating per-task weight updates changes where the task is learned; labeled evidence is still required. The literature develops two parallel approaches to this inference: native temporal in-context predictors and temporal features supplied to pretrained tabular in-context classifiers.

Pretraining on temporal classification tasks. TiCT, TIC-FM, and TimEE are representative approaches that learn to classify from labeled support [12,35,36]. TiCT and TimEE pretrain on synthetic classification tasks and make predictions from support without updating model weights for the target task. TIC-FM combines a pretrained temporal encoder with an in-context classifier. In these methods, pretraining is organized around complete classification tasks containing support examples and queries, rather than individual observations alone. This allows the model to learn how labeled examples specify the current classification task.

ChorusTIC [37] extends this setting to heterogeneous multivariate time series. Its synthetic tasks include class rules based on temporal patterns as well as relations across channels. Random Subchannel Slot Concatenation and a dual axis encoder allow the model to process different numbers of channels. The resulting features are then calibrated using the support set before prediction. This design extends temporal ICL beyond univariate classification and introduces relations across channels as part of the task definition. However, handling different channel counts does not by itself show that a model can infer a wide range of relational rules. This ability still depends on the task distribution seen during pretraining and on the information provided by the support examples.

Reuse of tabular ICL models. RocketPFN [15], MASHT [17], and TS2TabPFN [16] combine temporal feature extraction with a pretrained tabular ICL classifier. RocketPFN uses random convolutional features, MASHT combines MultiRocket and Hydra, and TS2TabPFN uses explicit temporal features. In each case, temporal information is first converted into features and is then processed by a tabular model that performs in-context classification. This design makes it possible to reuse an existing tabular task prior, but its performance still depends on whether the temporal features preserve the distinctions needed for the target task.

TSPFN [38] follows a different strategy. It starts from TabPFN, adds temporal and channel embeddings, and is further trained on labeled physiological data using an objective defined on query labels. For a new target task, prediction is again based on labeled support without updating model weights. In this case, temporal representation and in-context inference are trained within the same model. By contrast, methods such as RocketPFN and TS2TabPFN keep temporal feature extraction and in-context classification more separate. The two designs therefore differ in how closely temporal representation learning is tied to task inference.

Diversity of pretraining tasks. Diversity in pretraining observations is not the same as diversity in classification tasks. CauKer can generate varied synthetic time series for representation learning [11,45]. ICL models such as TiCT and TimEE must additionally learn how observations, support labels, and query labels are related within a task [12,35]. The synthetic causal prior diagnostic further motivates examining the diversity of these task structures [52].

A generator may produce many different waveforms while defining classes with the same underlying property. In-context classification, the model may instead need to infer different rules based on shape, temporal order, frequency, synchronization, phase, lag, correlation, or other relations across channels. Similar observations can therefore belong to different classification tasks when the rule connecting observations to labels changes. The support set helps identify that rule, but successful prediction also depends on whether the pretrained model has learned a sufficiently broad set of task

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

patterns. This motivates evaluating diversity at the level of classification tasks rather than only at the level of input observations.

The information available at inference time also matters. Feature transforms, scalers, retrieval procedures, and cached support representations may still be constructed for each target task. For example, the documented tsfresh configuration of TS2TabPFN uses both training data and unlabeled test covariates during feature extraction and imputation, making this configuration transductive [16]. TIC-FM aligns a shared projection adapter using UCR training splits before inference on each dataset, while TIC-FM (Syn.) performs this alignment using synthetic SCM data [36]. These operations should be recorded separately from the in-context prediction step because they change the information or preparation available before inference.

Current studies show that both native temporal ICL and the reuse of tabular ICL models can support time series classification. Some methods also extend to multivariate settings. Their generalization, however, depends on more than the diversity of input signals. It also depends on the range of classification tasks represented during pretraining, the information contained in the support set, and the amount of context the model can process. More support can reduce ambiguity, but it also increases memory and computation. Generalization to new task rules, reliable use of relations across channels, and scaling to larger class spaces therefore remain open questions.

ICL is especially useful when target tasks change frequently and can be specified through labeled support. It avoids updating the main model for every new task, but it does not remove the cost of preparing and processing the support set. Evaluation should therefore report both the labels available for the task and the labels actually processed for each prediction. Repeated support sampling is also useful for assessing stability. Sections 4.5 and 4.6 provide descriptive comparisons, while Section 5.3 discusses remaining questions about task coverage and scalability.

3.4. Comparison and Discussion

Mechanisms and practical tradeoffs. The three mechanisms differ in how target information is used. Representation transfer constructs a decision rule from fixed features, adaptation updates the pretrained model or selected modules, and ICL infers the target task from labeled support while keeping the main model fixed for each target task. These are alternative transfer strategies rather than stages of increasing capability. Table 3 summarizes their main differences.

Table 3. Design questions across the three transfer mechanisms. Target information enters at different locations, which changes the useful control experiment.

Mechanism	Target information enters through	Reusable knowledge	Main design question	Informative comparison
Representation transfer	A fitted rule or a target-derived feature transformation	Fixed pretrained encoder	Can the decision rule access the target distinction in these features?	Vary the head on identical features and target samples; test preprocessing separately
Fine-tuning and adaptation	Learned changes to the representation path and target head	Pretrained initialization and shared modules	Where must the representation change, and is there enough target evidence?	Match frozen and adapted runs; add the same architecture trained from random initialization
In-context learning	Labeled support and any constructed contextual state	A predictor trained across tasks	Can support specify the target rule within the learned prior and context capacity?	Fix the task pool, vary support size, repeat support sampling, and record processed context

The appropriate mechanism should be determined from evidence on the target task rather than from the backbone alone. For representation transfer, performance with a single classifier is insufficient to judge whether the representation is adequate. Comparing different classifiers on the same fixed features can help distinguish limited classifier access from limitations of the representation. If several

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

reasonable decision rules remain weak, while controlled encoder adaptation consistently improves performance, this provides stronger evidence that the representation itself needs to change. The scope of adaptation should then reflect the source of the mismatch, such as input handling, channel structure, or the representation itself.

For ICL, the main question is whether labeled support can reliably specify the target rule without task-specific parameter updates. Avoiding target-specific optimization can be useful when tasks change frequently or when on-device training is costly or unavailable. This advantage does not remove the cost of constructing and processing the support set. ICL should therefore be evaluated across support sizes, repeated support samples, and tasks whose class rules differ from those seen during pretraining.

Comparison of complete configurations. Reported performance reflects the complete classification procedure rather than the pretrained model alone. Feature construction, preprocessing, classifier choice, target data usage, and model selection can all affect the result. When the goal is to attribute a difference to one component, the remaining conditions should be controlled. For example, classifier comparisons should use the same features and target samples, while comparisons across backbones with a common classifier still retain differences in pretraining and feature construction.

These considerations motivate the empirical analysis in Section 4, which examines how benchmark conclusions vary with the target classification procedure, label budget, and task characteristics.

4. Benchmark and Empirical Analysis

We develop the TSC-FM Benchmark to provide a common empirical basis for comparing foundation models for time series classification. The benchmark records complete classification configurations rather than backbone names alone, since the same pretrained model can be used with different target procedures. The analysis in this section examines how the resulting conclusions depend on the target configuration, the available label budget, and the characteristics of the target tasks.[2]

The empirical scope covers frozen representation transfer and ICL on the reported task collection. Adaptation is not included in the benchmark comparisons considered here. The goal is therefore not to rank all transfer mechanisms, but to examine how stable the assessment of a pretrained system remains when the target procedure or evaluation setting changes.

4.1. Benchmark Design

The TSC-FM Benchmark contains 198 real-world time series classification datasets, each corresponding to one classification task. Among them, 128 are univariate and 70 are multivariate. By training set size, 58 datasets contain at most 100 examples, 82 contain between 101 and 500 examples, and 58 contain more than 500 examples. By sequence length, 74 datasets have at most 200 time steps, 61 have between 201 and 500 time steps, and 63 have more than 500 time steps. The benchmark includes 67 binary datasets, 96 datasets with three to ten classes, and 35 datasets with more than ten classes. Each dataset receives equal weight in the aggregate results.

A configuration is defined by how the pretrained model is used on the target task. This distinction allows the same backbone to appear with different target procedures. The benchmark therefore evaluates the resulting classification system rather than assigning one score to a backbone independently of its target classifier.

4.2. Evaluation Protocols

Standard evaluation uses the official training and test splits. Frozen-Probe fixes the encoder and trains a downstream classifier on the available training data, while ICL predicts from labeled support without updating the model parameters. Low-shot evaluation samples support examples by class from the training split.

[2] Results and protocol: https://tsc-fm.dmirlab.com/leaderboard and https://tsc-fm.dmirlab.com/evaluation. All analyses use the public aggregate snapshot retrieved on 13 September 2026.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

For a task set $\mathcal{D}$, performance is measured by the unweighted mean accuracy across tasks,

$$\bar{a}_m = \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} a_{m,d}.$$

Each task (dataset) therefore contributes equally to the Overall score. Since the benchmark contains 128 univariate and 70 multivariate datasets, the Overall score reflects these task counts rather than an equal average of the two subsets. Average Rank is reported separately and summarizes the relative position of each configuration across datasets.

Low-shot evaluation uses five or ten labels per class. A task is included only when every class contains at least the required number of training examples. Support examples are sampled by class and shared across methods. Results are compared only within the eligible task set for each budget. The five-label and ten-label settings therefore contain different task pools and should not be interpreted as a controlled comparison of the effect of additional labels.

4.3. Overall Performance

Table 4 reports the 12 configurations with complete coverage of the 198 Standard datasets. ChorusTIC achieves the highest mean accuracy at 78.56%, followed by MantisV2+LR at 77.73% and UniShape+RF at 76.87%. The difference between ChorusTIC and MantisV2+LR is 0.83 percentage points.

Table 4. Standard Overall of TSC-FM Benchmark. All 12 configurations cover the same 198 tasks. Accuracy is reported in percent; UV and MV use the 128 univariate and 70 multivariate tasks within this core. Text differences use unrounded values. Average Rank uses the 12 configurations, whereas the separate UV leaderboard contains additional configurations.

Configuration	Overall	Avg. rank	UV	MV
ChorusTIC	**78.56**	**3.94**	**81.16**	**73.78**
MantisV2+LR	77.73	4.64	80.03	73.51
UniShape+RF	76.87	5.08	78.86	73.22
MantisV2+RF	76.62	5.42	78.79	72.66
MOMENT+SVM	76.23	5.22	77.98	73.04
TabICLv2	76.10	4.89	78.88	71.02
MOMENT+RF	75.35	6.08	77.51	71.39
TabICL	73.83	5.99	76.84	68.33
MOMENT+kNN	72.01	7.74	75.72	65.24
NuTime+RF	66.76	8.95	69.39	61.95
MOMENT+NC	64.22	9.54	66.06	60.87
MOMENT+MLP	50.23	10.51	44.51	60.70

The upper part of the leaderboard is tightly clustered. The six highest scoring configurations range from 78.56% for ChorusTIC to 76.10% for TabICLv2, a span of 2.46 percentage points. These configurations include both ICL and frozen representation transfer. The Standard results therefore provide little evidence for a general accuracy advantage of one of these mechanisms over the other. Differences among particular implementations are of the same order as the differences observed across mechanisms.

The ordering also depends on how performance is aggregated. MOMENT+SVM has a slightly higher mean accuracy than TabICLv2, at 76.23% versus 76.10%, whereas TabICLv2 has the better average rank, at 4.89 versus 5.22. Thus, even with the task set held fixed, the ordering can change according to whether the comparison emphasizes average accuracy or relative position across datasets. A leaderboard position should therefore be interpreted together with the aggregation criterion used to obtain it.

Differences between leading configurations also change across task subsets. ChorusTIC exceeds MantisV2+LR by 1.13 percentage points on the univariate datasets, but by only 0.28 percentage points

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

on the multivariate datasets. The Overall score is useful as a common summary, but it does not describe the size of the advantage within every task regime.

4.4. Sensitivity to Target Configuration

Figure 2 shows that the same frozen representation can produce substantially different results under different downstream classifiers. For MOMENT, mean accuracy ranges from 76.23% with SVM to 50.23% with MLP, a difference of 26.00 percentage points.

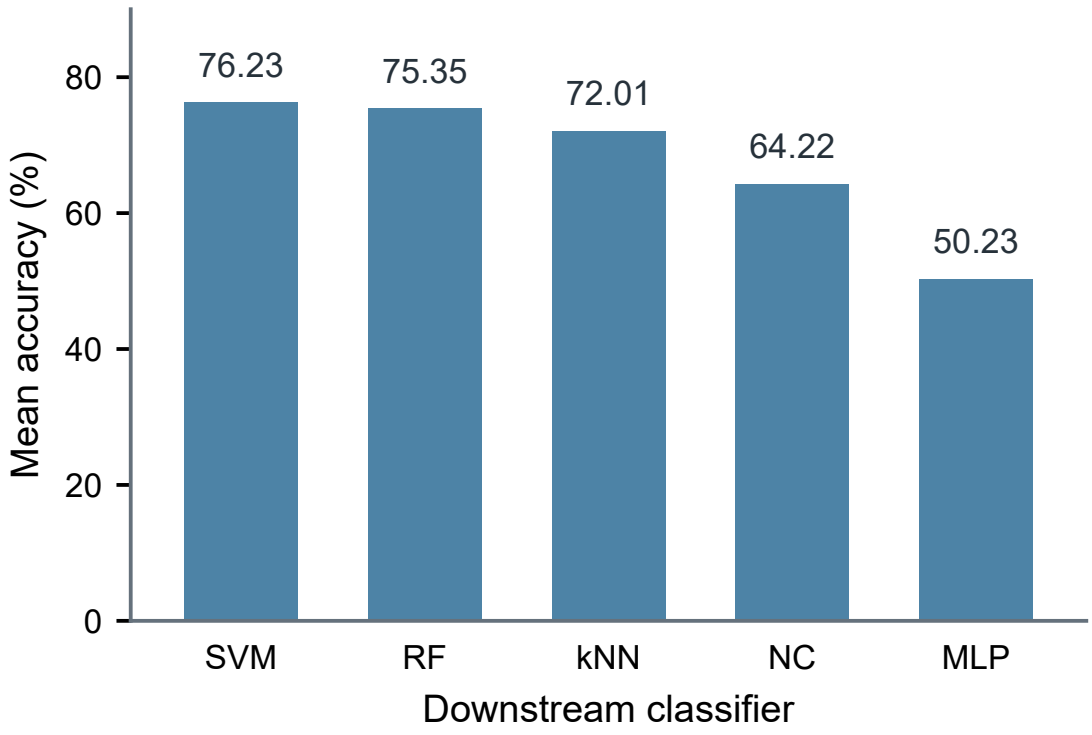


Figure 2. Standard configurations of MOMENT on the 198 datasets.

For MOMENT, these configurations use the same frozen representation and differ only in the downstream classifier. The observed variation therefore directly reflects the effect of the decision rule applied to the pretrained features. The benchmark performance of a frozen representation is thus not determined by the backbone alone.

A single classifier result is also insufficient to characterize representation quality. Poor performance may indicate that the classifier cannot effectively use information retained in the representation rather than that the information is absent. Controlled comparisons of different classifiers on the same features are therefore necessary when assessing frozen representation transfer.

4.5. Performance with Limited Labels

We evaluate Low-shot performance separately on univariate and multivariate datasets. This allows methods to be compared within the input settings they support, since models restricted to univariate inputs would otherwise be excluded by a requirement for multivariate coverage. It also avoids conflating predictive performance with differences in input compatibility.

As shown in Table 5, TIC-FM achieves the highest mean accuracy on univariate datasets at both label budgets, reaching 68.08% with five labels per class and 74.33% with ten. Its margins over the next best configuration are 0.54 and 0.96 percentage points, respectively. On multivariate datasets, Table 6 shows that ChorusTIC ranks first at both budgets, with mean accuracies of 62.61% and 68.05%. Its corresponding margins are 0.10 and 0.63 percentage points.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

Table 5. Univariate Low-shot mean accuracy (%). The 5-label-per-class view covers 112 eligible datasets and the 10-label-per-class view covers 97. All 19 configurations have complete coverage within the corresponding dataset pool. Results are ordered by the 5-label-per-class mean. Bold marks the best recorded mean within each view.

Configuration	5 per class ↑	10 per class ↑
TIC-FM	**68.08**	**74.33**
MantisV2+ScaledLR	67.54	73.30
Mantis+ScaledLR	67.44	73.37
Mantis+SVM	67.39	73.22
Mantis+LR	67.27	72.72
ChorusTIC	66.75	72.65
MantisV2+RF	66.09	71.65
Mantis+RF	65.14	71.05
MOMENT+RF	65.06	70.62
Mantis+kNN	65.06	69.85
NuTime+RF	64.97	70.44
MOMENT+SVM	64.96	71.48
MOMENT+kNN	64.17	68.69
Mantis+NC	62.89	66.80
TabICLv2	62.64	67.97
MOMENT+NC	61.75	63.96
TabICL	60.82	67.19
MOMENT+LR	59.63	63.61
MOMENT+MLP	47.70	49.88

Table 6. Multivariate Low-shot mean accuracy (%). The 5-label-per-class view covers 28 eligible UEA datasets and the 10-label-per-class view covers 24. All 18 configurations have complete coverage within the corresponding dataset pool. Results are ordered by the 5-label-per-class mean. Bold marks the best recorded mean within each view.

Configuration	5 per class ↑	10 per class ↑
ChorusTIC	**62.61**	**68.05**
Mantis+LR	62.51	65.57
TabICLv2	62.14	65.73
Mantis+ScaledLR	61.86	65.54
Mantis+SVM	61.54	67.42
MantisV2+ScaledLR	61.50	65.80
MantisV2+RF	60.15	63.49
Mantis+NC	60.08	64.07
Mantis+kNN	59.63	64.09
Mantis+RF	59.56	64.98
MOMENT+SVM	58.47	63.42
TabICL	58.17	64.60
MOMENT+RF	57.19	60.17
MOMENT+LR	55.85	60.13
MOMENT+NC	54.90	57.19
MOMENT+kNN	54.86	58.20
NuTime+RF	54.73	59.71
MOMENT+MLP	52.02	52.58

A consistent pattern emerges across the two input settings: an ICL configuration achieves the highest reported mean accuracy in all four Low-shot views. The margins are generally modest, so the results do not establish a broad dominance of ICL over other transfer mechanisms. They do, however, indicate that ICL is particularly competitive when only a small number of labeled target examples are available.

One possible explanation follows from where the target task is learned. Frozen representation methods must construct a new decision rule from the limited labeled target data, whereas ICL uses the support examples to specify the task to a predictor that has already been trained across classification tasks. Under small label budgets, the learned task prior may reduce the amount of target-specific

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

estimation required from the available labels. The observed results are consistent with this interpretation, but they do not isolate the mechanism causally because the compared systems also differ in pretraining data, representation, architecture, and input interface.

The results also show that the Low-shot strength of ICL is not yet unified in a single model across input settings. TIC-FM leads the univariate comparisons, whereas ChorusTIC leads the multivariate comparisons. This suggests that support-conditioned inference alone is not sufficient for broad transfer: the pretrained task prior and the input interface must also be compatible with the target setting. Current ICL methods therefore show strong performance under limited labels, while generalization across both univariate and multivariate classification remains incomplete.

4.6. Performance across Dataset Characteristics

Performance differences are not uniform across dataset groups. In the Standard comparison, MantisV2+LR achieves 80.03% on univariate datasets and 73.51% on multivariate datasets, while MOMENT+SVM obtains 77.98% and 73.04%, respectively. Their difference decreases from 2.06 percentage points on the univariate subset to 0.46 points on the multivariate subset. A similar pattern appears between ChorusTIC and MantisV2+LR, whose difference decreases from 1.13 to 0.28 percentage points. These comparisons show that an advantage observed on the full benchmark or on univariate datasets need not persist at the same magnitude on multivariate datasets. Since the two subsets contain different datasets, the result does not isolate channel count as the cause, but it does show that aggregate results can hide substantial variation across input settings.

The Capability Map provides a complementary view based on the number of classes.[3] We distinguish three method groups in this analysis. Native temporal ICL refers to temporal in-context predictors pretrained directly for time series classification from labeled support examples. Tabular ICL instead applies pretrained tabular in-context predictors to temporal features, while frozen representation transfer fits a downstream classifier on fixed pretrained features.

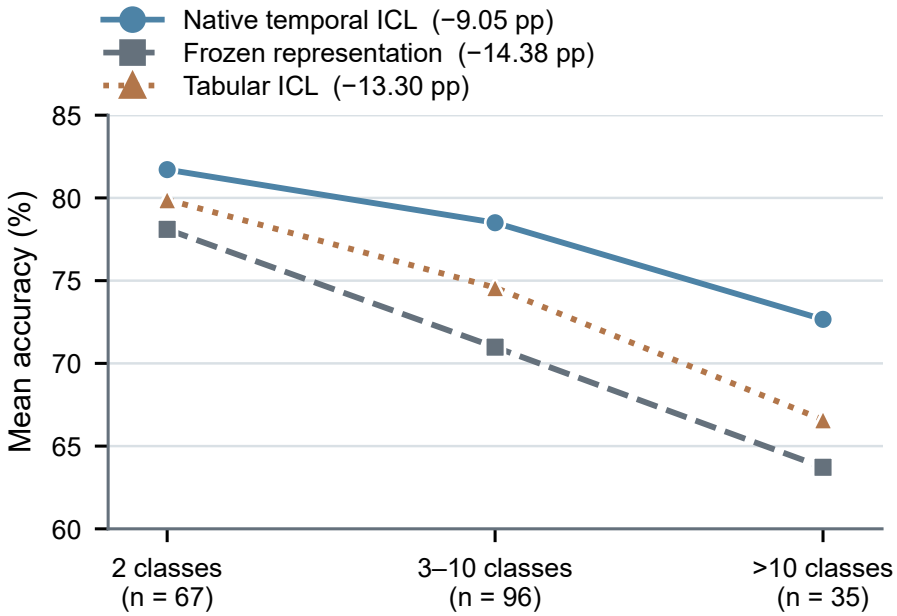


Figure 3. Mean accuracy across datasets with different numbers of classes. Configuration scores are first averaged within each backbone, after which backbones and datasets receive equal weight.

[3] https://tsc-fm.dmirlab.com/capability-map. Results were retrieved on 13 September 2026.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

A consistent decline appears as the number of classes increases. Native temporal ICL decreases from 81.71% on binary datasets to 78.50% on datasets with three to ten classes and 72.66% on datasets with more than ten classes. Frozen representation transfer decreases from 78.10% to 70.98% and then to 63.72%, while tabular ICL decreases from 79.89% to 74.58% and 66.59%. From binary datasets to those with more than ten classes, the corresponding decreases are 9.05, 14.38, and 13.30 percentage points.

More importantly, the three groups do not degrade at the same rate. Native temporal ICL shows the smallest decrease as the class space grows. Its advantage over frozen representation transfer increases from 3.61 percentage points on binary datasets to 7.52 points on datasets with three to ten classes and 8.94 points on datasets with more than ten classes. Its advantage over tabular ICL similarly increases from 1.82 to 3.92 and then to 6.07 percentage points. Larger class spaces therefore not only reduce accuracy, but also widen the performance differences among the reported method groups.

This pattern makes larger label spaces a particularly informative stress regime for current TSC foundation models. Binary datasets may conceal differences between transfer strategies because only a small number of class distinctions need to be established. As the number of classes increases, the target procedure must distinguish more classes from the available target information, making limitations in the learned representation or ICL task prior more visible.

The main empirical finding is not simply that datasets with more classes are harder. Rather, the reported method groups degrade at different rates as the class space grows. Class count thus provides a useful axis for revealing differences between transfer strategies and motivates controlled experiments that vary the size of the label space while keeping other dataset properties and label availability as fixed as possible.

4.7. Summary of Empirical Findings

The benchmark results lead to four main conclusions. First, the Standard comparison does not support a universal performance ordering between ICL and frozen representation transfer. Leading configurations from both mechanisms achieve similar aggregate accuracy, while mean accuracy and average rank can produce different local orderings even on the same datasets. The transfer mechanism alone therefore does not determine the leaderboard position; the complete target procedure also matters.

Second, frozen representation performance depends on the downstream decision rule. The reported MOMENT configurations produce substantially different accuracies when different classifiers are applied to the same pretrained representation. A frozen-feature result should therefore not be interpreted as a property of the backbone alone.

Third, performance under limited labels depends on the input setting. TIC-FM leads the univariate Low-shot comparison at both label budgets, whereas ChorusTIC leads the multivariate comparison. Both are native temporal ICL models, but they are designed for different input regimes. These results show that native temporal ICL models can be competitive under limited labels in both settings, while also indicating that strong ICL performance does not transfer automatically across input regimes. The pretrained task prior and the input interface remain important parts of the complete system.

Fourth, dataset characteristics can reveal differences that are obscured by aggregate accuracy. Performance gaps observed on univariate datasets become smaller in the reported multivariate comparisons. More notably, all three method groups lose accuracy as the number of classes increases, but they do so at different rates. In the current Capability Map, native temporal ICL shows a smaller decrease than frozen representation transfer and tabular ICL, and its performance advantage over the other two groups widens as the class space grows. Larger label spaces therefore provide a useful setting for distinguishing current transfer strategies rather than merely a uniformly harder version of binary classification.

Taken together, the empirical results support evaluating a foundation model through its complete classification configuration under a specified evaluation setting, rather than assigning a single performance value to the pretrained backbone. They also provide empirical counterparts to two distinctions developed in Section 3: representation quality cannot be separated from classifier access

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

without controlled comparisons, and ICL performance depends on the match between the learned task prior, the available support, and the target input setting. The benchmark does not currently include target adaptation, so the questions of update scope and task mismatch discussed in Section 3.2 remain supported by the literature rather than by the empirical comparisons in this section. These boundaries motivate the controlled evaluations discussed in Section 5.

5. Challenges and Future Directions

The literature reviewed in Section 3 shows that all three transfer mechanisms can support classification on new time series tasks. The main open questions are therefore no longer whether transfer is possible, but where each mechanism reaches its limits and how those limits can be identified. For representation transfer, the unresolved issue is which target distinctions remain accessible in fixed features. For adaptation, it is how to determine what should change from limited target evidence. For ICL, it is which classification rules can be inferred from labeled support and how far the learned task prior scales beyond the tasks seen during pretraining. The benchmark results in Section 4 further show that these questions cannot be separated from the target procedure and evaluation setting.

Table 7. Open problems across transfer mechanisms and the evidence needed to study them.

Goal	Current evidence	Open boundary	Required evaluation
Transferable representations	Frozen-feature configurations combine pretrained features with a target decision rule.	Which target distinctions survive the source objective, pooling, and channel processing?	Same-backbone classifier comparisons and controlled changes in domain, length, and channel structure.
Efficient adaptation	Full, partial, and parameter-efficient updates specialize different parts of the representation path.	Which update scope is useful under limited labels and preparation budgets?	Joint reporting of accuracy, label use, validation, optimization, setup time, and stored target state.
Generality	ICL models handle varying symbolic labels and reported archive tasks; synthetic task priors are emerging [12,35,36].	Which task mechanisms, class counts, domains, and label structures fall within the useful region of the learned task prior?	Controlled task families, label permutations, class-count transfer, fresh/source-held tasks, and mechanism-level OOD tests.
Generality: multivariate	Multichannel classification has been evaluated with different channel-processing designs [27,41,53].	Can context specify class rules defined by synchronization, phase, correlation, lag, or causal direction?	Channel permutation, independent/synchronous shifts, missing-channel tests, relation-defined synthetic tasks, and channel-count scaling.
Scalability	Reported ICL configurations use finite supports and class spaces; construction and repeated processing contribute to cost.	How do accuracy, memory, and latency change with support size, class count, sequence length, channels, and query volume?	Support/class-count curves, context truncation and retrieval disclosure, caching/compression ablations, and full-system cost curves.
Reliability	Accuracy alone leaves probability quality and failure recognition unresolved.	Does uncertainty rise when support is noisy, contradictory, shifted, or outside familiar task structure?	NLL/Brier/ECE, risk–coverage, conformal sets, label noise, contradictory support, open-set queries, and task-level mismatch analysis.
Contamination	Public archives are compact and repeatedly reused; synthetic-only pretraining reduces direct overlap but not benchmark-guided development.	Is apparent task generalization driven by memorized datasets, archive identity, or development leakage?	Internal provenance and duplicate checks, declared cutoffs, aggregate overlap reporting, and controlled source-held evaluation.
Reproducibility	Results remain sensitive to support construction, head/interface, preprocessing, model version, and system composition.	Can claims be reconstructed under the same target-task contract?	Controlled prediction and split records, checkpoint identifiers, and complete TargetFit fields with aggregate public summaries.

5.1. Learning Transferable Representations

Existing work has established that frozen representations learned from general time series data, classification-oriented objectives, synthetic data, other modalities, and domain-specific signals can support downstream TSC. The remaining question is more specific: which distinctions required by a new classification task are preserved in those representations?

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

This distinction matters because frozen-feature accuracy reflects both the representation and the decision rule used to interpret it. Section 4.4 shows that the same MOMENT representation produces substantially different results under different downstream classifiers. A weak probe therefore does not by itself show that the required information is absent. Conversely, increasing classifier capacity cannot recover information that has already been removed by pretraining, preprocessing, pooling, or channel aggregation. Future evaluation should therefore separate information retention from classifier access by varying the decision rule on identical features and target samples.

A second challenge is to determine which invariances are useful for future classification tasks. Removing amplitude variation, local timing differences, or channel identity may improve transfer when those factors are nuisance variables, but the same operation can be harmful when they define the target classes. The goal should therefore not be increasingly invariant representations in general, but representations that preserve a broad range of potentially discriminative temporal structure. Controlled task families in which waveform shape, amplitude, frequency, temporal order, or channel relations are varied independently would provide more direct evidence about what survives pretraining.

This shifts representation research from demonstrating broad reuse toward characterizing the information boundary of the pretrained encoder. Average downstream accuracy remains useful, but it should be complemented by tests that identify which class distinctions remain recoverable and which have been lost.

5.2. Selective and Efficient Target Adaptation

Adaptation research has moved beyond the choice between freezing a model and fine-tuning all of its parameters. Existing methods update input projections, prompts, channel embeddings, adapters, selected encoder layers, or the full backbone. These results establish that useful specialization can occur at several levels of the model. They do not yet provide a reliable rule for choosing among them.

The central problem is therefore update selection rather than parameter count alone. If the fixed representation already separates the target classes, changing the classifier may be sufficient. An input or channel mismatch may instead require an adapter, while a missing class distinction may require changes to the encoder itself. Updating fewer parameters restricts how much the model can change, but does not by itself reduce the need for target labels, validation, optimization, or checkpoint selection. Parameter efficiency should therefore be distinguished from preparation efficiency.

A useful direction is to treat adaptation scope as a target-dependent selection problem under an explicit budget. Comparisons should start from the same frozen configuration and ask whether progressively broader updates provide reproducible gains with the available target evidence. The relevant cost should include not only trainable parameters, but also labels used for fitting and validation, optimization time, model selection, and stored target-specific state. Such comparisons are especially important under small label budgets, where a more flexible update can improve task fit but also increase overfitting and negative transfer.

The longer-term challenge is to diagnose the mismatch before choosing the update. Current methods largely specify the adaptation scope in advance. A stronger adaptation system would determine whether the target problem lies mainly in the input interface, channel handling, representation, or decision rule and update only the components needed to correct that mismatch.

5.3. General and Scalable In-Context Learning

Native temporal ICL and the reuse of tabular ICL models have shown that a new TSC task can be specified through labeled support without updating the main predictor for each target task [12,15–17,35]. The Low-shot results in Section 4.5 further show that ICL can be competitive when target labels are scarce. However, the leading model differs between univariate and multivariate settings, suggesting that support-conditioned inference alone is not sufficient for broad transfer. The learned task prior and the input interface must also match the target setting.

A central challenge is therefore the coverage of the pretrained task prior. Diversity in input signals does not imply diversity in classification rules. A model may observe many different time series

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

during pretraining while repeatedly learning similar class distinctions. General ICL should instead cover a broader range of rules based on shape, frequency, temporal order, phase, lag, synchronization, correlation, and their combinations. Evaluation should likewise test target rules that differ from those used during pretraining, rather than only new datasets generated from familiar task structures.

Multivariate classification makes this issue more demanding. Accepting different numbers of channels does not show that a model can infer class rules defined by relations between channels. Tasks based on synchronization, phase, lag, or correlation require the predictor to use relational information rather than only evidence from individual channels. Controlled evaluations that alter cross-channel relations while preserving the information within each channel would provide more direct evidence of this capability.

Scalability is another open problem. Section 4.6 shows that performance decreases as the number of classes increases, indicating that larger class spaces provide a useful test of ICL task inference. At the same time, larger support sets can reduce ambiguity but increase memory and inference cost. Retrieval, caching, compression, prototypes, and latent summaries may reduce repeated processing, but can also remove evidence needed to identify the target rule. Future evaluation should therefore study accuracy and cost jointly across class count, support size, sequence length, channel count, and query volume.

5.4. Reliable Transfer under Task Mismatch

The three transfer mechanisms fail in different ways when the target evidence is weak or mismatched. A fitted classifier can overfit limited labels, adaptation can move the representation in the wrong direction, and ICL can infer an incorrect task from noisy or contradictory support. Reliability therefore requires more than high average accuracy: a system should also respond appropriately when the information available for establishing the target classifier is insufficient.

Distribution shift should be analyzed according to whether it changes the classification rule. A change in sensor scale or sampling conditions may preserve the target distinction, whereas a missing informative channel or a new class can change the information required for prediction. These cases should not be grouped under a single notion of robustness. For adaptation, methods should be compared with the corresponding fixed model under each specified shift, as illustrated by NeuroAdapt-Bench [51]. For ICL, label permutation, corrupted associations, contradictory support, support order, and truncation provide complementary tests of whether prediction actually follows the supplied task information [52].

Reliability should also include uncertainty about whether the current task lies within the useful transfer region. Probability metrics such as negative log-likelihood, Brier score, and calibration error can complement accuracy, while risk–coverage analysis can evaluate whether the model abstains on uncertain examples or unfamiliar tasks. Conformal prediction provides another route to set-valued predictions, although its validity depends on the calibration data and assumptions under distribution shift [54–56].

A useful future system may therefore combine mechanisms rather than apply one unconditionally. A confident frozen classifier or ICL predictor could be used directly, while insufficient evidence could trigger additional support, retrieval, or target adaptation. Such fallbacks should not be treated as free additions: their label use, computation, and stored target state become part of the complete classification procedure.

5.5. Reproducible and Contamination-Aware Evaluation

Benchmark conclusions depend on the complete classification configuration, not only on the pretrained backbone. Classifier choice, preprocessing, label budget, support construction, and aggregation can all affect the reported result. Reproducible evaluation should therefore specify the target procedure, data split, checkpoint, validation use, and any target-specific state.

Low-shot evaluation requires particular care. Different label budgets can change the set of eligible datasets, so aggregate differences across budgets do not directly measure label efficiency. Controlled

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

comparisons should use the same eligible datasets and, when support is sampled, report results over repeated support sets.

Pretraining overlap is another important source of uncertainty. Public TSC datasets are repeatedly used for pretraining, development, and evaluation, while synthetic pretraining does not eliminate the possibility that benchmark performance influences model design. Claims about generalization to unseen tasks therefore require clear records of source data, development cutoffs, duplicate checks, and evaluation splits. Future benchmarks should make these conditions explicit so that gains from pretrained knowledge can be distinguished from differences in evaluation procedure.

6. Conclusions

Foundation models provide reusable knowledge for time series classification, but that knowledge must still be connected to the labels and discriminative structure of a new target task. This survey organizes existing approaches according to where this target task is learned. Representation transfer constructs a decision rule on fixed pretrained features, fine-tuning and adaptation modify the pretrained model or selected components using target data, and in-context learning infers the target rule from labeled support without updating the main predictor for each task. These mechanisms provide complementary ways to reuse pretrained knowledge rather than a hierarchy of increasing capability. Their effectiveness depends on what information is preserved during pretraining, what can be learned from the available target evidence, and how the model handles the target input setting.

The TSC-FM Benchmark complements this synthesis by examining complete classification configurations across different evaluation settings. The results do not support a universal performance ordering between frozen representation transfer and ICL. They instead show that the assessment of a pretrained system can change with the downstream decision rule, label budget, input setting, and dataset characteristics. Native temporal ICL is competitive under limited labels, but the leading model differs between univariate and multivariate settings, while larger class spaces reveal different rates of performance degradation across method groups. These findings reinforce the need to evaluate the complete target procedure rather than treating performance as a property of the pretrained backbone alone.

Future progress therefore depends on moving beyond demonstrations that transfer is possible toward understanding when and why it succeeds. Important directions include identifying which class distinctions survive pretraining, selecting the appropriate scope of target adaptation, broadening the task priors learned by ICL models, and scaling in-context learning to richer multivariate relations, larger class spaces, and larger support sets. Reliable transfer also requires handling distribution shifts that preserve the target task and recognizing changes that alter the classification rule itself, while reproducible evaluation requires explicit target procedures, data splits, model versions, and source exposure. Together, these directions suggest that future progress will depend on both expanding the effective range of each transfer mechanism across new task rules and input conditions, and developing reliable ways to recognize when a model is operating outside that range.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

Appendix A. Model and Evaluation Details

Table A1. Illustrative TARGETFIT ledger entries. Rows instantiate the six audit fields under representative target-task contracts; they are not additional transfer mechanisms or a ranking of methods.

Illustrative contract	D_{used}	Y_{used}	Θ_{updated}	H: deployed rule	V: selection	B: material budget
Representation transfer: fitted classifier	Training; validation if used	Training labels; validation labels if used	Classifier parameters; encoder fixed	Fitted classifier on fixed representations	Layer, pooling, or classifier hyperparameters	Feature extraction, classifier fitting, and search trials
Adaptation: PEFT or adapter	Training; validation if used	Training labels; validation labels if used	Adapter and head parameters; statistics if updated	Prediction head over adapted network state	Checkpoint, early-stopping rule, or hyperparameters	Backward passes, validation, and search
Representation transfer: prototype control	Labeled reference set	Reference labels	Class prototypes; no learned target parameters	Nearest-prototype rule with stated metric	Layer, metric, or reference subset if selected	Encoding, prototype construction, and distance computation
In-context learning: support-conditioned predictor	Labeled support and query inputs; query batch if jointly processed	Support labels; validation labels if used	No target-specific weight update; fitted transforms or cached state if present	Pretrained predictor conditioned on labeled support	Retrieval, context order, or ensemble if selected	Transform construction if used, context encoding, and forward passes

Table A2. Representative target procedures retained from the existing method records. Mechanism labels follow the three-mechanism taxonomy; multiple assignments require explicitly supported, evaluated use modes of the proposed system in different mechanisms, excluding baseline-only and ablation-only configurations. The status and boundary refer to the cited public version.

System	Transfer mecha-nism(s)	Target labels	Target fitting / updates	Updated / constructed state	Status	Supported claim and boundary
MOMENT [9]	Rep. transfer	Yes	SVM fit only	Target RBF-SVM; backbone frozen	Journal / main conf.	Transferable features under a selected nonlinear target classifier; the cited record evaluates frozen-feature TSC.
Mantis [11]	Rep. transfer + adaptation	Yes	Mode-dependent	LR/RF, or all layers plus head	Journal / main conf.	CauKer-synthetic-only pretraining supports broad transfer; calibration is unreported in the final paper.
UniShape [42]	Adaptation + rep. transfer	Yes	Yes in main interface	Encoder, adapter, head; held-out RF alternative	Journal / main conf.	Shape-aware pretraining supports univariate transfer; main interface is end-to-end fine-tuning, not prototype inference.
UniTS [10]	Adaptation	Yes	Yes	Full model, prompt tokens, task tokens, or class representations	Journal / main conf.	Full and lightweight adaptation are both evaluated; they have different task-establishment budgets.
FORMED [27]	Adaptation	Yes	Yes	Channel embeddings and label queries for new target	Journal / main conf.	Two-stage medical repurposing is feasible; adaptation is not label-query-only.
TiCT / TimEE [12,35]	In-context	Support	No	Labeled support; TiCT may use retrieval/cache state; no target weights	Preprints	Native support-conditioned TSC is evaluated; both remain recent and depend on support protocol.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

Table A2. *Cont.*

System	Transfer mecha-nism(s)	Target labels	Target fitting / updates	Updated / constructed state	Status	Supported claim and boundary
TIC-FM [36]	In-context	Support	No per-task gradients	Labeled support context	Workshop	Deployment-time inference avoids task-specific classifier fitting and parameter updates. The default adapter uses UCR training splits, while TIC-FM (Syn.) uses synthetic SCM data.
RocketPFN [15]	In-context	Support	No learned weights	ROCKET kernel bank and support context	Preprint	Tabular-PFN composition is feasible and may still construct target-specific state.
TS2TabPFN / MASHT [16,17]	In-context	Support	No learned weights	Fitted temporal transforms, encoders, and context	Preprints	Uni-/multivariate PFN composition is feasible; released pipelines perform target fit/state construction.
TiViT [13]	Rep. transfer	Yes	Probe-dependent	Logistic/RF or class centroids; ViT frozen	Preprint	Vision features support TSC and provide a centroid control; target labels and selection remain material.

Table A3. Reporting dimensions and additional evaluation diagnostics for TSCFMs. These describe evidence requirements, not completed benchmark runs.

Protocol item	Why it matters	Recommended reporting practice
Pretraining/benchmark provenance	Archive or near-duplicate exposure invalidates an unseen-task interpretation.	Retain internal manifests and overlap records; report verified aggregate exposure, exclusions, and cutoffs within the access policy.
Backbone update	"Frozen" and fine-tuned systems have different data and compute budgets.	State full tune, PEFT, adapter, batch-norm/statistic update, or strictly frozen.
Target-label use (TARGETFIT)	A linear/SVM/logistic head is supervised target training even with a frozen encoder.	Report algorithm, hyperparameter search, validation source, seeds, fit time, and number of labels.
Meaning of zero-shot	The term conflates feature transfer, frozen backbones, absent optimization, and absent labels.	Use the mechanism definitions in Section 2.3; never use "training-free" when a target classifier is fitted.
Few-shot construction	Unstratified samples can omit classes; per-class budgets and total budgets are not equivalent.	State shots per class/total, sampling, eligibility and task counts at each budget. Compare label gains on shared tasks; retain sampled indices internally.
Support/query protocol	ICL predictions may depend on order, label coding, query batching, and support truncation.	Report support seeds/order, query batch composition, context length, label permutations, and aggregation.
Task-prior coverage	Archive accuracy alone leaves generalizable task mechanisms unidentified.	Report pretraining task structure where known; evaluate controlled changes in class count, task mechanism, domain, and multivariate relations.
Support/class-count scaling	ICL quality and cost can change sharply as support and label spaces grow.	Report accuracy, memory, truncation/retrieval behavior, and latency across several support sizes and class counts.
Archive aggregation	Mean accuracy weights small and large datasets equally and hides task families.	Retain paired task scores internally; publish aggregate or permitted group statistics with coverage and missing-run rules.
Resampling and uncertainty	A single official split can have high variance.	For Low-shot, repeat support sampling with the test split fixed. Report model seeds separately; treat alternative train/test resamples as a distinct protocol.
Statistical comparison	Many pairwise claims inflate family-wise error.	Use justified paired analyses and multiplicity control; report comparison units, effect estimates, and treatment of ties.
Calibration/reliability	Similar accuracy can conceal unusable probabilities.	Report NLL, multiclass Brier score, ECE with bin definition, reliability plots, and any calibration-set consumption.
Multivariate stress tests	Standard UEA scores do not isolate cross-channel reasoning.	Add channel permutation, missing channels, independent versus synchronous shifts, and channel-count scaling.
Efficiency and composition	Ensembles and target fitting can dominate backbone inference.	Report fit/inference wall time, peak memory, FLOPs or MACs, activated parameters, context length, and ensemble/model count.
Code and artifacts	Hidden preprocessing and version drift complicate reproduction.	Pin checkpoint, commit, environment, preprocessing, splits, configs, and prediction files.

Appendix A.1. Model Configurations

The TARGETFIT notation records target information and operations:

$$\text{TARGETFIT} = (D_{\text{used}}, Y_{\text{used}}, \Theta_{\text{updated}}, H, V, B).$$

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

D_{used} identifies accessed target partitions, including unlabeled query batches if they jointly influence prediction. Y_{used} records which labels are used. Θ_{updated} records learned updates and constructed statistics or cached state, whose types must be distinguished. H is the deployed decision rule, V records validation and selection, and B records computational budgets. The notation is an accounting device and does not create another method taxonomy.

For adaptation, the record needs to identify the updated modules and selection procedure. UniTS [10], multivariate adapters [41], LLMFew [14], and FORMED [27] illustrate different update scopes. Their target costs cannot be reduced to trainable-parameter counts. For in-context prediction, TiCT [12] and TimEE [35] illustrate native support-conditioned configurations, while RocketPFN [15], TS2TabPFN [16], and MASHT [17] illustrate compositions that can construct transforms or contextual state. The default TIC-FM [36] projection alignment is recorded separately from its fixed-parameter per-task inference. These examples explain the fields; they do not assert inclusion in the accompanying benchmark.

Table A1 contains illustrative accounting entries, and Table A2 retains the existing representative method records. Neither table is a benchmark performance table. Actual benchmark configuration records need to identify the checkpoint, preprocessing, target procedure, version, and run status for each included system. Literature tables describe the cited paper's evaluated modes, whereas benchmark rows describe the website's configurations. For example, the UniShape paper record covers univariate evaluation, while the website reports UniShape+RF on multivariate tasks. The benchmark wrapper's channel handling remains to be documented; its reported score does not establish a native multivariate interface in the cited model.

Appendix A.2. Dataset Composition and Selection

Section 4.1 reports the public profile of 198 tasks and their aggregate attributes. Individual task identities, source mappings, and task-level result mappings are withheld. The published aggregates describe collection composition but do not independently establish selection, deduplication, or paired task coverage. Those properties require the underlying task records. Shot eligibility is determined by the number of training examples available in every class, as described in Section 4.5.

Appendix A.3. Data Splits and Pretraining Overlap

Source exposure, model development, and target evaluation need separate records. Benchmark task identities being hidden does not establish that a model was never pretrained or developed on them. Forecasting benchmarks illustrate related leakage and partition risks [57,58]. Synthetic-only training can reduce direct use of real samples, but archive-guided generator design, architecture selection, and checkpoint choice remain separate possible exposure paths. NuTime [29], for example, reports pretraining on pooled training portions of UCR, UEA, and other datasets. Its reported archive classification results therefore do not establish transfer to wholly unseen datasets. Relating this source exposure to the benchmark configuration additionally requires the actual checkpoint and task records.

The available materials do not establish the absence or extent of benchmark overlap. Resolving it requires source manifests, duplicate checks, exclusions, development cutoffs, and evaluation splits. Withheld task identities further restrict external verification. Conclusions in Section 4 therefore concern performance on the reported collection, without an unseen-task or contamination-free interpretation.

Appendix B. Evaluation Reporting and Access

Appendix B.1. Reporting Dimensions

Table A3 summarizes the reporting dimensions derived from the method comparison and the open problems in Section 5. It distinguishes configuration documentation from additional evaluation diagnostics. Statistical comparisons need paired records and a missing-run policy; resampling intervals need repeated observations. The table is a guide for interpreting and extending evaluations, not a record of completed benchmark experiments.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

Appendix B.2. Evaluation Access and Result Versions

The accompanying website, https://tsc-fm.dmirlab.com/, provides aggregate results, method and checkpoint links, a capability view, and a form for registering public code and checkpoint repositories. The submission page describes metadata registration without execution of submitted model files. The website is described here as a result and model-registration resource; automated evaluation and independent reruns are not claimed.

This paper analyzes the public snapshot retrieved on 13 September 2026 under the displayed protocol v0.1. The saved source pages and their hashes identify the aggregates used for the tables. The capability payload also identifies its aggregate data version. These records fix the analyzed webpage content; they do not supply immutable versions of all underlying runs. The website may subsequently change, so its live ranking should be distinguished from the snapshot analyzed here.

Refreshed evaluation offers one way to study later tasks under declared development cutoffs, with forecasting providing a related precedent [59]. For TSC, the support/query definition and symbolic label mapping remain necessary parts of that design. Any future access or rerun procedure should state what an external user can verify and what remains inaccessible under the hidden-task policy.

References

1. Hannun, A.Y.; Rajpurkar, P.; Haghpanahi, M.; Tison, G.H.; Bourn, C.; Turakhia, M.P.; Ng, A.Y. Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network. *Nature medicine* **2019**, *25*, 65–69.
2. Ince, T.; Kiranyaz, S.; Eren, L.; Askar, M.; Gabbouj, M. Real-time motor fault detection by 1-D convolutional neural networks. *IEEE Transactions on Industrial Electronics* **2016**, *63*, 7067–7075.
3. Ordóñez, F.J.; Roggen, D. Deep convolutional and lstm recurrent neural networks for multimodal wearable activity recognition. *Sensors* **2016**, *16*, 115.
4. Garnot, V.S.F.; Landrieu, L.; Giordano, S.; Chehata, N. Satellite image time series classification with pixel-set encoders and temporal self-attention. In Proceedings of the 2020 IEEE/CVF conference on computer vision and pattern recognition (CVPR). IEEE, 2020, pp. 12322–12331.
5. Ismail Fawaz, H.; Forestier, G.; Weber, J.; Idoumghar, L.; Muller, P.A. Deep learning for time series classification: a review. *Data Mining and Knowledge Discovery* **2019**, *33*, 917–963.
6. Bagnall, A.; Dau, H.A.; Lines, J.; Flynn, M.; Large, J.; Bostrom, A.; Southam, P.; Keogh, E. The UEA multivariate time series classification archive, 2018. *arXiv preprint arXiv:1811.00075* **2018**.
7. Bagnall, A.; Lines, J.; Bostrom, A.; Large, J.; Keogh, E. The Great Time Series Classification Bake Off: A Review and Experimental Evaluation of Recent Algorithmic Advances. *Data Mining and Knowledge Discovery* **2017**, *31*, 606–660.
8. Foumani, N.M.; Tan, C.W.; Webb, G.I.; Salehi, M. Improving position encoding of transformers for multivariate time series classification. *arXiv preprint arXiv:2305.16642* **2023**.
9. Goswami, M.; Szafer, K.; Choudhry, A.; Cai, Y.; Li, S.; Dubrawski, A. MOMENT: A Family of Open Time-series Foundation Models. In Proceedings of the Proceedings of the 41st International Conference on Machine Learning. PMLR, 2024, Vol. 235, *Proceedings of Machine Learning Research*, pp. 16115–16152.
10. Gao, S.; Koker, T.; Queen, O.; Hartvigsen, T.; Tsiligkaridis, T.; Zitnik, M. UniTS: A Unified Multi-Task Time Series Model. In Proceedings of the Advances in Neural Information Processing Systems, 2024, Vol. 37, pp. 140589–140631.
11. Feofanov, V.; Wen, S.; Xie, S.; Roschmann, S.; Alonso, M.; Guo, H.; Ilbert, R.; Tiomoko, M.; Bouniot, Q.; Akata, Z.; et al. Mantis: Lightweight Foundation Model for Time Series Classification. In Proceedings of the Proceedings of the 43rd International Conference on Machine Learning, 2026, Vol. 306, *Proceedings of Machine Learning Research*.
12. Yeh, C.C.M.; Saini, U.S.; Wang, J.; Dai, X.; Fan, X.; Sun, J.; Fan, Y.; Zheng, Y. TiCT: A Synthetically Pre-Trained Foundation Model for Time Series Classification. *arXiv preprint arXiv:2511.19694* **2025**.
13. Roschmann, S.; Bouniot, Q.; Feofanov, V.; Redko, I.; Akata, Z. TiViT: Time Series Representations for Classification Lie Hidden in Pretrained Vision Transformers. *arXiv preprint arXiv:2506.08641* **2025**.
14. Chen, Y.; Li, Z.; Yang, C.; Wang, X.; Xu, G. Large Language Models Are Few-Shot Multivariate Time Series Classifiers. *Data Mining and Knowledge Discovery* **2025**, *39*, 66.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

15. O'Rourke, F.M.; Trisovic, A.; Bertsimas, D. RocketPFN: Accurate Time Series Classification via In-Context Learning. *arXiv preprint arXiv:2606.21786* **2026**.
16. Merlin, G.d.C.; Silva, D.F. TS2TabPFN: Time Series Classification and Extrinsic Regression through Feature Extraction and a Tabular Foundation Model. *arXiv preprint arXiv:2608.04174* **2026**.
17. Cüppers, J.; Vreeken, J. In-Context Time Series Classification with Random Convolutional Features. *arXiv preprint arXiv:2607.19234* **2026**.
18. Foumani, N.M.; Miller, L.; Tan, C.W.; Webb, G.I.; Forestier, G.; Salehi, M. Deep Learning for Time Series Classification and Extrinsic Regression: A Current Survey. *ACM Computing Surveys* **2024**, *56*, 1–45.
19. Zhang, K.; Wen, Q.; Zhang, C.; Cai, R.; Jin, M.; Liu, Y.; Zhang, J.Y.; Liang, Y.; Pang, G.; Song, D.; et al. Self-Supervised Learning for Time Series Analysis: Taxonomy, Progress, and Prospects. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **2024**, *46*, 6775–6794.
20. Ma, Q.; Liu, Z.; Zheng, Z.; Huang, Z.; Zhu, S.; Yu, Z.; Kwok, J.T. A Survey on Time-Series Pre-Trained Models. *IEEE Transactions on Knowledge and Data Engineering* **2024**, *36*, 7536–7555.
21. Liang, Y.; Wen, H.; Nie, Y.; Jiang, Y.; Jin, M.; Song, D.; Pan, S.; Wen, Q. Foundation Models for Time Series Analysis: A Tutorial and Survey. In Proceedings of the Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 6555–6565.
22. Ye, J.; Yu, Y.; Zhang, W.; Wang, L.; Li, J.; Tsung, F. Empowering Time Series Analysis with Foundation Models: A Comprehensive Survey. *Information Fusion* **2027**, *137*, 104601.
23. Kottapalli, S.R.K.; Hubli, K.; Chandrashekhara, S.; Jain, G.; Hubli, S.; Botla, G.; Doddaiah, R. Foundation Models for Time Series: A Survey. *arXiv preprint arXiv:2504.04011* **2025**.
24. Liu, Z.; Li, B.; Huang, H.; Sun, Y.; Wang, Y.; Wu, M.; Ma, Q. From Pre-training to Post-training: A Survey on Time Series Foundation Models. *TechRxiv* **2026**, pp. 1–23.
25. Trirat, P.; Shin, Y.; Kang, J.; Nam, Y.; Na, J.; Bae, M.; Kim, J.; Kim, B.; Lee, J.G. Universal Time-Series Representation Learning: A Survey. *ACM Computing Surveys* **2026**, *58*, 1–40.
26. Babu, N.; Mathew, J.; Vinod, A.P. Large language models for time-series analysis: A survey with experimental evaluation for classification. *Expert Systems with Applications* **2027**, *333*, 133932. https://doi.org/10.1016/j.eswa.2026.133932.
27. Huang, N.; Wang, H.; He, Z.; Zitnik, M.; Zhang, X. Repurposing Foundation Model for Generalizable Medical Time Series Classification. In Proceedings of the International Conference on Learning Representations, 2026.
28. Li, Z.; Natarajan, K.; Zhang, W.; Zhou, M.; Lee, S.A.; Zhang, Y.; Xu, M.A.; Esmaeilpour, Z.; Salim, F.D.; Malhotra, M.; et al. GlucoFM: A Dual-Stream Foundation Model for Continuous Glucose Monitoring. *arXiv preprint arXiv:2605.30865* **2026**, [2605.30865].
29. Lin, C.; Wen, X.; Cao, W.; Huang, C.; Bian, J.; Lin, S.; Wu, Z. NuTime: Numerically Multi-Scaled Embedding for Large-Scale Time-Series Pretraining. *Transactions on Machine Learning Research* **2024**.
30. Ismail-Fawaz, A.; Devanne, M.; Berretti, S.; Weber, J.; Forestier, G. Finding foundation models for time series classification with a pretext task. In Proceedings of the Pacific-Asia Conference on Knowledge Discovery and Data Mining. Springer, 2024, pp. 123–135.
31. Dani, M.; Liebe, S. Parameter-Efficient Self-Supervised Adaptation for EEG-FM under Fixed Computational Budgets. *arXiv preprint arXiv:2608.24727* **2026**, [2608.24727].
32. Quinlan, P.; Li, Q.; Zhu, X. ADAPTive Input Training for Many-to-One Pre-Training on Time-Series Classification. In Proceedings of the Proceedings of the 39th Canadian Conference on Artificial Intelligence. PMLR, 2026, Vol. 318, *Proceedings of Machine Learning Research*, pp. 296–307.
33. Li, Z.; Wang, B.; Ke, Z. From Tabular to Time Series: Can TabPFN Handle Mixed Data? A Study on PhysioNet. In Proceedings of the ICML Workshop on Foundation Models for Structured Data, 2025.
34. Tegon, A.; Lehmann, N.; Li, Y.; Cossettini, A.; Benini, L.; Ingolfsson, T.M. FEMBA on the Edge: Physiologically-Aware Pre-Training, Quantization, and Deployment of a Bidirectional Mamba EEG Foundation Model on an Ultra-low Power Microcontroller. *arXiv preprint arXiv:2603.26716* **2026**, [2603.26716].
35. Küken, J.; Hoo, S.B.; Mráz, M.; Hutter, F.; Purucker, L. TimEE: End-to-end Time Series Classification via In-Context Learning. *arXiv preprint arXiv:2607.07500* **2026**.
36. Fang, J.; Xie, S.; Nie, S.; Ling, Y.; Liu, Y.; Li, Z.; Zhang, K.; Pan, L.; Palpanas, T.; Cai, R. Beyond Task-Specific Classifiers: In-Context Inference for Time Series Classification Foundation Models. In Proceedings of the 2nd ICML Workshop on Foundation Models for Structured Data, 2026.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

37. Fang, J.; Xie, S.; Cai, R.; Zheng, S.; Li, Z.; Zhang, K.; Pan, L.; Palpanas, T.; Hao, Z. ChorusTIC: Training-Free Multivariate Time Series Classification via Chorus In-Context Learning. *arXiv preprint arXiv:2608.24033* **2026**, [2608.24033].
38. Stym-Popper, J.; Rambour, C.; Granese, F.; Thome, N.; Bernard, O. TSPFN: A Temporal Tabular Foundation Model for Physiological Time Series Classification. *arXiv preprint arXiv:2608.31013* **2026**, [2608.31013].
39. Quinlan, P.; Levasseur, J.; Li, Q.; Zhu, X. Chronicle: A Multimodal Foundation Model for Joint Language and Time Series Understanding. *arXiv preprint arXiv:2605.20268* **2026**.
40. Feofanov, V.; Wen, S.; Zhang, J.; Pan, L.; Redko, I. MantisV2: Closing the Zero-Shot Gap in Time Series Classification with Synthetic Data and Test-Time Strategies. *arXiv preprint arXiv:2602.17868* **2026**.
41. Ilbert, R.; Feofanov, V.; Tiomoko, M.; Redko, I.; Palpanas, T. User-Friendly Foundation Model Adapters for Multivariate Time Series Classification. In Proceedings of the 2025 IEEE 41st International Conference on Data Engineering Workshops, 2025, pp. 136–144.
42. Liu, Z.; Wang, Y.; Li, B.; Zheng, J.; Eldele, E.; Wu, M.; Ma, Q. A Unified Shape-Aware Foundation Model for Time Series Classification. In Proceedings of the Proceedings of the AAAI Conference on Artificial Intelligence, 2026, Vol. 40, pp. 23972–23980.
43. Moakher, Y.; Attia El Hili, Y.; Feofanov, V. UTICA: Multi-Objective Self-Distllation Foundation Model Pretraining for Time Series Classification. *arXiv preprint arXiv:2603.01348* **2026**. Also presented at the ICLR 2026 Workshop on Time Series in the Age of Large Models.
44. Dou, Y.; Wu, H.; Chen, G.; Ju, F.; Cui, Z.; Shen, D. Taming foundation model with invariance-oriented pre-training for broad-spectrum EEG analysis across signal-level, brain-state, and brain-health tasks. *arXiv preprint arXiv:2608.24597* **2026**, [2608.24597].
45. Xie, S.; Feofanov, V.; Alonso, M.; Odonnat, A.; Zhang, J.; Redko, I. Cauker: classification time series foundation models can be pretrained on synthetic data only. In Proceedings of the 1st ICML Workshop on Foundation Models for Structured Data, 2025.
46. Auer, A.; Klotz, D.; Böck, S.; Hochreiter, S. Pre-trained Forecasting Models: Strong Zero-Shot Feature Extractors for Time Series Classification. *arXiv preprint arXiv:2510.26777* **2025**.
47. Schumacher, D.; Nourbakhsh, E.; Slavin, R.; Rios, A. Prompting Underestimates LLM Capability for Time Series Classification. *arXiv preprint arXiv:2601.03464* **2026**.
48. Qiu, C.; Xu, Y.; Wang, Y.; Bagnall, A. Local Reference Geometry Residual Augmentation for Imbalanced Time Series Classification. *arXiv preprint arXiv:2609.00093* **2026**, [2609.00093].
49. Grill, J.B.; Strub, F.; Altché, F.; Tallec, C.; Richemond, P.; Buchatskaya, E.; Doersch, C.; Avila Pires, B.; Guo, Z.; Gheshlaghi Azar, M.; et al. Bootstrap your own latent-a new approach to self-supervised learning. *Advances in neural information processing systems* **2020**, *33*, 21271–21284.
50. Nguyen, A.T.; Sun, Z.; Johnson, M.J. Lightweight Adaptation of EEG Foundation Models for Stroke Motor Imagery Decoding: Domain Shift and Subject-Level Robustness. *arXiv preprint arXiv:2609.00282* **2026**, [2609.00282].
51. Lee, G.J.; Pradeepkumar, J.; Sun, J. Test-Time Adaptation for EEG Foundation Models: A Systematic Study under Real-World Distribution Shifts. *arXiv preprint arXiv:2604.16926* **2026**, [2604.16926].
52. Cai, H.R.; Ye, H.J. Synthetic Causal Priors for In-Context Time-Series Classification. In Proceedings of the ICML Workshop on Foundation Models for Structured Data, 2026.
53. Sungu Isiacik, P.; Nguyen, T.L.; Aderinola, T.; Ifrim, G. An empirical evaluation of foundation models for multivariate time series classification. In Proceedings of the Joint European Conference on Machine Learning and Knowledge Discovery in Databases. Springer, 2025, pp. 3–21.
54. Brier, G.W. Verification of Forecasts Expressed in Terms of Probability. *Monthly Weather Review* **1950**, *78*, 1–3.
55. Guo, C.; Pleiss, G.; Sun, Y.; Weinberger, K.Q. On Calibration of Modern Neural Networks. In Proceedings of the Proceedings of the 34th International Conference on Machine Learning. PMLR, 2017, Vol. 70, *Proceedings of Machine Learning Research*, pp. 1321–1330.
56. Nanopoulos, A.; Buza, K. Conformal Prediction for Out-of-Distribution Time-Series Classification. *Applied Intelligence* **2025**, *55*, 823.
57. Aksu, T.; Woo, G.; Liu, J.; Liu, X.; Liu, C.; Savarese, S.; Xiong, C.; Sahoo, D. GIFT-Eval: A Benchmark for General Time Series Forecasting Model Evaluation. *arXiv preprint arXiv:2410.10393* **2024**.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 24 September 2026 doi:10.20944/preprints202609.2097.v1

58. Meyer, M.; Kaltenpoth, S.; Zalipski, K.; Müller, O. Time series foundation models: Benchmarking challenges and requirements. *arXiv e-prints* **2025**, pp. arXiv–2510.
59. Meyer, M.; Kaltenpoth, S.; Albers, H.; Zalipski, K.; Müller, O. TS-Arena -A Live Forecast Pre-Registration Platform. In Proceedings of the Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2, 2026, pp. 9558–9568.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.